

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ”
ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ
Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №2

Специальность ПО-13

Выполнил:

А.В. Шевчук

студент группы ПО-13

Проверил:

А. Д. Кулик

Брест 2026

Лабораторная работа No 2

Цель работы: закрепить навыки объектно-ориентированного программирования на языке Python

Задание 1

Реализовать простой класс

Требования к выполнению

- Реализовать пользовательский класс по варианту.

Для каждого класса

- Создать атрибуты (поля) классов
- Создать методы классов
- Добавить необходимые свойства и сеттеры (по необходимости)
- Переопределить магические методы `__str__` и `__eq__`

7) Множество символов ограниченной мощности – Предусмотреть возможность объединения двух множеств, вывода на консоль элементов множества, а также метод, определяющий, принадлежит ли указанное значение множеству. Класс должен содержать методы, позволяющие добавлять и удалять элемент в/из множества. Конструктор должен позволять создавать объекты с начальной инициализацией. Мощность множества задается при создании объекта. Реализацию множества осуществить на базе списка. Переопределить метод `__eq__`, выполняющий сравнение объектов данного типа.

Код программы:

```
class LimitedCharSet:
    """Represent a set of unique one-character strings with fixed capacity."""

    def __init__(self, capacity: int, initial: list[str] | None = None) -> None:
        """Initialize the set with capacity and optional initial data."""
        if capacity <= 0:
            raise ValueError("Capacity must be a positive integer.")

        self._capacity = capacity
        self._items: list[str] = []

        if initial is not None:
            for item in initial:
                self.add(item)

    @property
    def capacity(self) -> int:
        """Return maximum number of elements in the set."""
        return self._capacity
```

```

@capacity.setter
def capacity(self, new_capacity: int) -> None:
    """Set a new capacity if it is valid for current content."""
    if new_capacity <= 0:
        raise ValueError("Capacity must be a positive integer.")
    if new_capacity < len(self._items):
        raise ValueError("New capacity cannot be smaller than current set size.")

    self._capacity = new_capacity

def _validate_symbol(self, symbol: str) -> None:
    """Validate that value is a single character string."""
    if not isinstance(symbol, str):
        raise TypeError("Symbol must be a string.")
    if len(symbol) != 1:
        raise ValueError("Symbol must contain exactly one character.")

def add(self, symbol: str) -> bool:
    """Add symbol to the set.

    Return True if symbol was added, otherwise False.
    """
    self._validate_symbol(symbol)

    if symbol in self._items:
        return False

    if len(self._items) >= self._capacity:
        raise ValueError("Set capacity exceeded.")

    self._items.append(symbol)
    return True

def remove(self, symbol: str) -> bool:
    """Remove symbol from the set.

    Return True if symbol existed and was removed, otherwise False.
    """
    self._validate_symbol(symbol)

    if symbol not in self._items:
        return False

```

```

        self._items.remove(symbol)
        return True

def contains(self, symbol: str) -> bool:
    """Return whether the set contains the given symbol."""
    self._validate_symbol(symbol)
    return symbol in self._items

def union(self, other: "LimitedCharSet") -> "LimitedCharSet":
    """Return union of two character sets as a new object."""
    if not isinstance(other, LimitedCharSet):
        raise TypeError("Union is supported only for LimitedCharSet objects.")

    combined_items = self._items.copy()
    for item in other._items:
        if item not in combined_items:
            combined_items.append(item)

    result_capacity = max(self.capacity, other.capacity, len(combined_items))
    return LimitedCharSet(result_capacity, combined_items)

def get_elements(self) -> list[str]:
    """Return a copy of set elements."""
    return self._items.copy()

def __str__(self) -> str:
    """Return human-readable representation of the set."""
    content = ", ".join(self._items)
    return f"LimitedCharSet(capacity={self.capacity}, elements=[{content}])"

def __eq__(self, other: object) -> bool:
    """Compare two sets by capacity and content."""
    if not isinstance(other, LimitedCharSet):
        return False

    return self.capacity == other.capacity and sorted(self._items) == sorted(other._items)

def main() -> None:
    """Demonstrate class usage."""
    first_set = LimitedCharSet(5, ["a", "b", "c"])
    second_set = LimitedCharSet(5, ["c", "d"])

    print("First set:", first_set)
    print("Second set:", second_set)

```

```

print("Add 'e' to first set:", first_set.add("e"))
print("Remove 'b' from first set:", first_set.remove("b"))
print("Contains 'a':", first_set.contains("a"))
print("Contains 'z':", first_set.contains("z"))

united_set = first_set.union(second_set)
print("Union:", united_set)

another_set = LimitedCharSet(5, ["a", "c", "e"])
print("First set == another set:", first_set == another_set)

if __name__ == "__main__":
    main()

```

```

\python.exe c:/Users/user/Documents/GitHub/spp_po13/reports/Shevchuk/2/src/1.py
First set: LimitedCharSet(capacity=5, elements=[a, b, c])
Second set: LimitedCharSet(capacity=5, elements=[c, d])
Add 'e' to first set: True
Remove 'b' from first set: True
Contains 'a': True
Contains 'z': False
Union: LimitedCharSet(capacity=5, elements=[a, c, e, d])
First set == another set: True

```

Задание 2

Построить модель программной системы с применением отношений (обобщения, агрегации, ассоциации, реализации) между классами. Задать атрибуты и методы классов. Реализовать (если необходимо) дополнительные классы. Продемонстрировать работу разработанной системы.

7) Система Автобаза. Диспетчер распределяет заявки на Рейсы между Водителями и назначает для этого Автомобиль. Водитель может сделать заявку на ремонт. Диспетчер может отстранить Водителя от работы. Водитель делает отметку о выполнении Рейса и состоянии Автомобиля.

Код программы:

```

from __future__ import annotations

from abc import ABC, abstractmethod
from dataclasses import dataclass

class Person(ABC):
    """Base class for people in the system."""

```

```

def __init__(self, name: str) -> None:
    """Initialize person with name."""
    if not name.strip():
        raise ValueError("Name cannot be empty.")
    self.name = name

def __str__(self) -> str:
    """Return string representation."""
    return f'{self.__class__.__name__}(name='{self.name}')
```

```

def __eq__(self, other: object) -> bool:
    """Compare people by type and name."""
    return isinstance(other, self.__class__) and self.name == other.name
```

```

class Vehicle(ABC):
    """Abstract base class for depot vehicles."""

    def __init__(self, vehicle_id: str, model: str) -> None:
        """Initialize vehicle."""
        if not vehicle_id.strip():
            raise ValueError("Vehicle id cannot be empty.")
        if not model.strip():
            raise ValueError("Model cannot be empty.")

        self.vehicle_id = vehicle_id
        self.model = model
        self.is_available = True
        self.needs_repair = False

    @abstractmethod
    def vehicle_type(self) -> str:
        """Return vehicle type."""

    def mark_repair_needed(self) -> None:
        """Mark vehicle as requiring repair."""
        self.needs_repair = True
        self.is_available = False

    def mark_serviceable(self) -> None:
        """Mark vehicle as ready for work."""
        self.needs_repair = False
        self.is_available = True
```

```

def __str__(self) -> str:
    """Return string representation."""
    return (
        f'{self.vehicle_type()}(id={self.vehicle_id}, model={self.model}, '
        f'available={self.is_available}, needs_repair={self.needs_repair})'
    )

def __eq__(self, other: object) -> bool:
    """Compare vehicles by type and identifier."""
    return isinstance(other, self.__class__) and self.vehicle_id == other.vehicle_id

```

```

class Car(Vehicle):
    """Concrete vehicle used for trips."""

```

```

    def vehicle_type(self) -> str:
        """Return vehicle type."""
        return "Car"

```

```

class WorkerAction(ABC):
    """Interface-like abstract class for worker actions."""

```

```

    @abstractmethod
    def perform_action(self) -> str:
        """Perform and describe an action."""

```

```

@dataclass
class TripRequest:
    """Represent a request for a trip."""

```

```

    route_name: str
    cargo_description: str
    assigned: bool = False

```

```

@dataclass
class RepairRequest:
    """Represent a driver's repair request."""

```

```

    driver_name: str
    vehicle_id: str
    description: str
    completed: bool = False

```

```

class Trip(WorkerAction):
    """Represent a trip assigned to a driver and a vehicle."""

```

```

def __init__(self, request: TripRequest, driver: Driver, vehicle: Vehicle) -> None:
    """Initialize trip."""
    self.request = request
    self.driver = driver
    self.vehicle = vehicle
    self.completed = False
    self.vehicle_condition = "unknown"

```

```

def complete(self, vehicle_condition: str) -> None:
    """Complete the trip and save vehicle condition."""
    self.completed = True
    self.vehicle_condition = vehicle_condition
    self.vehicle.is_available = True

```

```

def perform_action(self) -> str:
    """Describe trip execution."""
    return (
        f'Trip on route '{self.request.route_name}' is assigned to {self.driver.name} '
        f'using vehicle {self.vehicle.vehicle_id}.'
    )

```

```

def __str__(self) -> str:
    """Return string representation."""
    return (
        f'Trip(route='{self.request.route_name}', driver='{self.driver.name}', '
        f'vehicle='{self.vehicle.vehicle_id}', completed={self.completed}, '
        f'condition='{self.vehicle_condition}')"
    )

```

```

def __eq__(self, other: object) -> bool:
    """Compare trips by request, driver and vehicle."""
    if not isinstance(other, Trip):
        return False

    return (
        self.request.route_name == other.request.route_name
        and self.driver == other.driver
        and self.vehicle == other.vehicle
    )

```

```

class Driver(Person):
    """Represent a driver."""

```



```

def __init__(self, name: str) -> None:
    """Initialize driver."""
    super().__init__(name)
    self.is_suspended = False
    self.current_trip: Trip | None = None

def request_repair(self, vehicle: Vehicle, description: str) -> RepairRequest:
    """Create a repair request for a vehicle."""
    if not description.strip():
        raise ValueError("Repair description cannot be empty.")

    vehicle.mark_repair_needed()
    return RepairRequest(self.name, vehicle.vehicle_id, description)

def assign_trip(self, trip: Trip) -> None:
    """Assign trip to driver."""
    if self.is_suspended:
        raise ValueError("Suspended driver cannot be assigned to a trip.")
    self.current_trip = trip

def complete_trip(self, vehicle_condition: str) -> None:
    """Complete current trip."""
    if self.current_trip is None:
        raise ValueError("Driver has no assigned trip.")

    self.current_trip.complete(vehicle_condition)
    self.current_trip = None

def __str__(self) -> str:
    """Return string representation."""
    return f'Driver(name='{self.name}', suspended={self.is_suspended})'

class Dispatcher(Person):
    """Represent a dispatcher."""

    def assign_trip(
        self,
        request: TripRequest,
        driver: Driver,
        vehicle: Vehicle,
    ) -> Trip:
        """Assign request to a driver and a vehicle."""
        if request.assigned:
            raise ValueError("Trip request is already assigned.")

```

```

    if driver.is_suspended:
        raise ValueError("Driver is suspended.")
    if not vehicle.is_available or vehicle.needs_repair:
        raise ValueError("Vehicle is not available.")

    request.assigned = True
    vehicle.is_available = False
    trip = Trip(request, driver, vehicle)
    driver.assign_trip(trip)
    return trip

def suspend_driver(self, driver: Driver) -> None:
    """Suspend a driver from work."""
    driver.is_suspended = True

def restore_driver(self, driver: Driver) -> None:
    """Restore a driver to work."""
    driver.is_suspended = False

def __str__(self) -> str:
    """Return string representation."""
    return f'Dispatcher(name='{self.name}')"

class BusDepot:
    """Represent bus depot system."""

    def __init__(self, name: str) -> None:
        """Initialize depot."""
        if not name.strip():
            raise ValueError("Depot name cannot be empty.")

        self.name = name
        self.drivers: list[Driver] = []
        self.vehicles: list[Vehicle] = []
        self.trip_requests: list[TripRequest] = []
        self.repair_requests: list[RepairRequest] = []
        self.trips: list[Trip] = []

    def add_driver(self, driver: Driver) -> None:
        """Add driver to depot."""
        if driver not in self.drivers:
            self.drivers.append(driver)

    def add_vehicle(self, vehicle: Vehicle) -> None:

```

```

        """Add vehicle to depot."""
        if vehicle not in self.vehicles:
            self.vehicles.append(vehicle)

def add_trip_request(self, request: TripRequest) -> None:
    """Add trip request."""
    self.trip_requests.append(request)

def add_repair_request(self, request: RepairRequest) -> None:
    """Add repair request."""
    self.repair_requests.append(request)

def add_trip(self, trip: Trip) -> None:
    """Add trip to history."""
    self.trips.append(trip)

def __str__(self) -> str:
    """Return string representation."""
    return (
        f"BusDepot(name='{self.name}', drivers={len(self.drivers)}, "
        f"vehicles={len(self.vehicles)}, trip_requests={len(self.trip_requests)}, "
        f"repair_requests={len(self.repair_requests)}, trips={len(self.trips)})"
    )

def __eq__(self, other: object) -> bool:
    """Compare depots by name."""
    return isinstance(other, BusDepot) and self.name == other.name

def main() -> None:
    """Demonstrate bus depot system."""
    depot = BusDepot("City Bus Depot")
    dispatcher = Dispatcher("Ivan Petrov")
    driver = Driver("Sergey Ivanov")
    vehicle = Car("A-101", "MAZ-203")

    depot.add_driver(driver)
    depot.add_vehicle(vehicle)

    trip_request = TripRequest("Minsk-Brest", "Passengers")
    depot.add_trip_request(trip_request)

    trip = dispatcher.assign_trip(trip_request, driver, vehicle)
    depot.add_trip(trip)

```

```

print(depot)
print(dispatcher)
print(driver)
print(vehicle)
print(trip.perform_action())

driver.complete_trip("Serviceable")
print("Trip completed.")
print(vehicle)

repair_request = driver.request_repair(vehicle, "Brake system inspection required")
depot.add_repair_request(repair_request)

print("Repair request created:")
print(repair_request)

dispatcher.suspend_driver(driver)
print("Driver suspended:", driver)

if __name__ == "__main__":
    main()

```

```

\python.exe c:/Users/user/Documents/GitHub/spp_po13/reports/Shevchuk/2/src/2.py
BusDepot(name='City Bus Depot', drivers=1, vehicles=1, trip_requests=1, repair_requests=0, trips=1)
Dispatcher(name='Ivan Petrov')
Driver(name='Sergey Ivanov', suspended=False)
Car(id='A-101', model='MAZ-203', available=False, needs_repair=False)
Trip on route 'Minsk-Brest' is assigned to Sergey Ivanov using vehicle A-101.
Trip completed.
Car(id='A-101', model='MAZ-203', available=True, needs_repair=False)
Repair request created:
RepairRequest(driver_name='Sergey Ivanov', vehicle_id='A-101', description='Brake system inspection required', completed=False)
Driver suspended: Driver(name='Sergey Ivanov', suspended=True)

```

Вывод: Я закрепил навыки объектно-ориентированного программирования на языке Python.